

815agent 完整报告

最小完备 Coding Agent · 模块设计 / 如何运行 / 与 kimi-code & Hermes 对比 / 污染分析与问题修复 ·
2026-08-15

[概览](#)[模块设计](#)[如何运行](#)[对比报告](#)[问题与解决](#)[工程问答](#)

概览

📖 设计依据原文: [《Agent 模块化设计教程 v2》](#) (52 页, kimi-code × Hermes 双 codebase 全模块对照)—— 本项目的每个模块都对应其中一章的最小设计。

这是什么

815agent 是一个**能自动写代码的 Agent Harness**: 把一个只会生成 token 的模型 (默认 qwen3.7-max, 30 万上下文), 变成一个能读文件、跑命令、记教训、守边界的工程实体。核心哲学来自教程的两条设计宪法:

宪法一 · Prompt Cache 神圣 前缀只追加不修改; 动态内容 (记忆、steer) 一律注入消息尾部。

宪法二 · 窄腰核心 核心 = 一个循环 + 一个消息列表 + 一个工具调用协议; 沙盒/记忆/技能/Hooks/ACP 全部挂在边上。

三层架构

表面层 cli.py (终端交互/一次性任务) · acp.py (stdio JSON-RPC 宿主)

Harness loop.py (窄腰核心 Agent: while ask→act→append)

└ context.py	每步 preflight 压缩（85% 阈值/头尾保留/免疫包裹）
└ tools.py	10 工具注册表 + 校验 + 50K 落盘治理
└ sandbox.py	hardline 硬线 → 去混淆 → 三分级裁决 → workdir 围栏
└ hooks.py	5 事件回调表 + 进程外 hook（30s 强杀）+ stop 闷锁
└ memory.py	MEMORY.md 冻结快照 + provenance 追加 + 注入安检
└ skills.py	SKILL.md 索引常驻一行 + 正文按需注入
└ subagent.py	delegate_task 隔离派生（独立上下文/预算/transcript）
└ session.py	JSONL transcript 追加重放（崩溃原子/坏行容错）

模型层 llm.py → qwen3.7-max（OpenAI 兼容，结构化重试 429/5xx/超时）

验证证据（全部现场实测）

验证	结果
离线单元测试（FakeLLM，覆盖全部模块）	35 / 35 OK（本机 + Docker 内一致）
现场写项目 taskcli（待办 CLI）	18 tests OK，agent 自愈空 JSON bug 并沉淀教训
Docker 内现场写项目 expense-cli	15 tests OK，干净容器复跑一致
5 个复杂压力任务（T1-T5）	39/19/54/26 tests OK，73 次工具调用 0 错误
hook 阻断适配探针	pip install 被进程外 hook 阻断 → 模型不重试、改标准库完成
大文件结果治理	241KB 日志 → 模型只见 2,130 字符 preview，全量 255,935 字符落盘
--resume 会话恢复	重放后模型准确回忆上轮文件名与统计结果
上下文压缩实战（小窗口强制）	压缩真实触发 2 次，免疫包裹注入，任务完成
delegate_task 子代理	父子 transcript 分离；子代理 11 次探索调用仅回传蒸馏报告
密钥卫生	子进程环境剥离 KEY/TOKEN 类变量；工作区零密钥痕迹

项目位置

本地： `D:\Users\liu.liu\Desktop\815agent\`（源码 + DESIGN.md + 本报告 report/）

dev-server: Docker 镜像 `815agent:latest` (构建于 /tmp/815agent_build/, 现场产物 /tmp/815agent_ws/)

模块设计

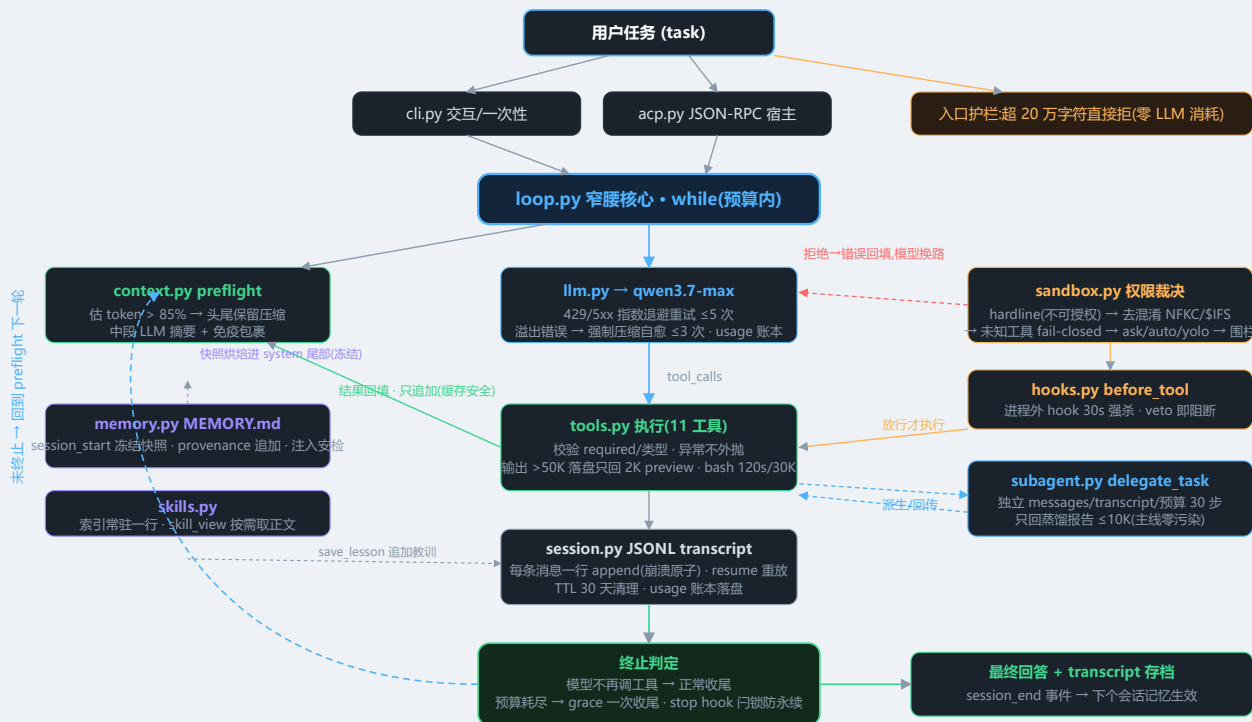
总览：模块分级表

分级含义： `L0 骨架` 教程最小设计，能跑 · `L1 健壮` 失败路径处理 · `L2 生产化` 生产事故反推的机制 · `L3 演化` 对照 kimi/Hermes 的下一级（本仓库按需取 1~2 级，不强上）

#	模块	行数	职责一句话	当前分级	防什么事故（痛点）	参照物
1	<code>loop.py</code>	210	主循环： ask→act→append,唯一的窄腰核心	<code>L2</code>	死循环烧钱、半途崩溃丢进度、行为不可回放	kimi 主循环形状
2	<code>context.py</code>	118	每步 preflight 上下文压缩(85% 阈值)	<code>L2</code>	长任务 token 溢出报错即死;粗暴截断打碎缓存/协议对	kimi 阈值触发+头尾保留
3	<code>tools.py</code>	298	11 工具注册表 + 校验 + 结果治理	<code>L2</code>	单条工具输出灌爆 30 万窗口(T5 实测 24 万字符)	kimi 截断+落盘
4	<code>sandbox.py</code>	117	权限裁决:hardline→去混淆→三分级→围栏	<code>L3 最小版</code>	rm -rf /、混淆绕过、越权读写、密钥外泄	Hermes hardline 先于 yolo
5	<code>memory.py</code>	100	MEMORY.md 冻结快照 + 原子追加 + 注入安检	<code>L2</code>	并发写截断竞态、毒记忆劫持每个新会话	kimi AppendLogStore / Hermes 安检

6	<code>skills.py</code>	86	SKILL.md 索引一行常驻,正文按需取	L1	提示词膨胀撑爆前缀缓存	kimi 三层披露
7	<code>hooks.py</code>	133	5 事件回调表 + 进程外 hook + stop 门锁	L2	hook 崩溃连坐主循环、stop 永续阻断死锁	kimi stopHook 门锁
8	<code>acp.py</code>	104	stdio JSON-RPC 分发 (宿主接入面)	L1	一条 print 混入 stdout 毁掉协议流	ACP 协议
9	<code>session.py</code>	81	JSONL transcript 追加重放 + TTL 清理	L2	崩溃丢会话;transcript 无限增长	kimi append-only 日志
10	<code>subagent.py</code>	35	delegate_task 隔离派生子代理	L1	探索过程污染主线上上下文;父子并发压缩分叉	kimi 扁平子代理 / Hermes #38727
11	<code>llm.py</code>	114	OpenAI 兼容客户端 + 退避重试 + 溢出自愈	L2	429/5xx 抖动、arguments 解析失败、上下文溢出即死	kimi 溢出自愈上限 3 次
12	<code>cli.py</code>	107	交互/一次性/ask 三形态 + Windows 编码修复	L1	GBK 控制台乱码、无恢复入口	—

整体 workflows (知识图谱)



815agent 整体工作流:蓝=主线数据流·绿=上下文/结果治理·橙=安全裁决链·红=拒绝路径·虚线紫=会话边界机制·虚线蓝=循环回边。矢量图,浏览器缩放不糊,可右键另存。

逐模块详述

1. 主循环 `loop.py` (210 行) L2

教程 Ch.1 · ask → act → append

- **四条不变量**：消息列表是唯一状态；终止由模型决定（不调工具即结束）；工具错误作为文本回填让模型自愈；一切变更只追加（缓存宪法）。
- **演化级 2/5**： `AGENT815_MAX_STEPS` （默认 60）预算硬墙兜底，耗尽后给一次 grace 收尾调用；重试下沉到 `llm.py`。
- **装配点**： `Agent.__init__` 绑定 `sandbox/memory/skills/hooks/transcript`，其余模块全部挂在这个类的边上。

2. 上下文 `context.py` (118 行) L2

教程 Ch.2 · 阈值触发 + 头尾保留 + 中段摘要

- chars/4 粗估；超 85% `AGENT815_MAX_CONTEXT`（默认 30 万）触发，未触发原样返回（缓存安全）。
- head = system + 首条 user；tail 从尾部倒走 ≤20% 预算；**不切 tool_call/result 对**；最后一条 user 必须在 tail（且已修复"单 user 会话被守护规则误杀"的边界，见问题页）。
- 中段 LLM 摘要以 `[CONTEXT COMPACTION - REFERENCE ONLY]` 免疫包裹 + "最新用户消息优先"；摘要失败 fail-open 不阻塞主循环。

3. 工具 `tools.py` (298 行) L2

教程 Ch.3 · schema + handler 字典注册表

- 10 个内置工具：`read_file / list_dir / grep / glob / write_file / edit_file / bash / skill_view / save_lesson / delegate_task`。
- `execute()` 做 required/类型校验，异常不外抛（"工具错误也是结果"）。
- **结果治理（演化级 2）**：输出超 50K 字符全量落盘 `.agent815/tool_results/`，模型只见 2K preview + 路径；bash 输出 30K 截断。

4. 沙盒权限 `sandbox.py` (117 行) L3 最小版

教程 Ch.4 · 审批回调三分级 + hardline 先于一切授权

- 裁决顺序：**hardline** (`rm -rf /`、`mkfs`、`dd` 裸设备、**fork bomb**、**写 authorized_keys** 等，**yolo 也不放行**) → 未知工具 fail-closed → 只读放行 → ask/auto/yolo 按模式。
- **去混淆（演化级 4 最小版）**：NFKC 全角折叠 + `$IFS / ${IFS}` 折叠 + 引号拼接（`r'm`）剥离后再匹配。
- 写工具路径强制 `workdir` 围栏；bash 子进程环境剥离 KEY/TOKEN/PASSWORD/SECRET/CREDENTIAL 类变量。

5. 记忆 `memory.py` (100 行,已加固) L2

教程 Ch.5 · MEMORY.md 冻结快照 + 追加写

- 会话开始读全文 **烘焙进 system prompt 尾部后冻结**（前缀缓存全程命中）；会话中 `save_lesson` 追加写，下个会话生效。

- **多会话防污染三件套**：追加带 UTC provenance（行级原子）；超限截断走 tmp+rename 原子替换；快照加载逐行**注入安检**，命中 promptware 模式的条目替换为占位符。

6. 技能 `skills.py` (86 行) L1

教程 Ch.6 · SKILL.md frontmatter 索引 + 延迟绑定

- 扫描项目 `skills/` 与 workdir `.agent815/skills/`；frontmatter (name/description) 常驻 system prompt 一行索引。
- 正文按需：`skill_view(name)` 工具取全文 + 资源路径提示（三层披露的最小两层）。

7. Hooks `hooks.py` (133 行) L2

教程 Ch.7 · 回调表 + 阻断配额

- 5 事件：session_start / session_end / before_tool / after_tool / stop；**只有 before_tool 和 stop 可阻断**。
- 进程外 hook (`hooks.d/<event>_*.py|sh`)：子进程运行、stdin 传 JSON payload、30s 超时强杀、stdout JSON 裁决；崩溃不影响主循环（隔离）。
- **stop 续跑一次性门锁**：防"hook 永远 block → turn 永不结束"（kimi stopHookContinuationUsed 的等价物）。

8. ACP `acp.py` (104 行) L1

教程 Ch.8 · stdio JSON-RPC 分发表

- 方法：initialize / session/new / session/prompt；未知带 id 方法显式 -32601；notification 静默不回复。
- **stdout 只走协议帧**，一切日志重定向 stderr（一条 print 混进 stdout = 协议流损坏）。

9. 会话持久化 `session.py` (81 行) L2

教程 Ch.9 · JSONL transcript 追加重放

- 每条消息/事件一行 JSON，append 天然崩溃原子（半行写坏只丢最后一行）；resume = 重放，坏行容错跳过。

- 子代理 transcript 独立文件 —— 多子代理隔离的物理基础。

10. 子代理 `subagent.py` (35 行,新增) L1

教程 Ch.9 · 对照 kimi-code 扁平子代理

- `delegate_task`：派生全新 Agent 实例 —— 独立 messages / 独立 transcript / 独立预算（默认 30 步） / 独立 stop 锁。
- **父压缩物理上碰不到子上下文**（不共享任何可变状态）；只回传最终文本（distill, 10K 截断），探索过程不灌主线窗口。
- 子代理 system prompt 与父一致（同前缀，暖缓存）。

附属:LLM 客户端 `llm.py` (114 行) L2 • CLI `cli.py` (107 行) L1

- `llm.py`: OpenAI 兼容 chat; `enable_thinking:false` (qwen3 系必须)；429/5xx/超时指数退避 ×2 上限 30s + 20% 抖动 + 尊重 Retry-After, 最多 5 次; arguments JSON 解析失败降级为空对象 + error 标记。
- `cli.py`: 交互 / 一次性 / ask 三形态; `/exit /memory /compact`; --resume 重放; 启动即 stdout/stderr reconfigure utf-8 (Windows GBK 修复)。

如何运行

1. 安装

```
cd 815agent
pip install -r requirements.txt # 只有 requests
```

2. 配置 (环境变量)

变量	说明	默认
AGENT815_API_KEY	LLM API key (必填, 运行时注入)	—
AGENT815_BASE_URL	OpenAI 兼容接口	https://dashscope.aliyuncs.com/compatible-mode/v1
AGENT815_MODEL	模型名	qwen3.7-max
AGENT815_WORKDIR	沙盒围栏根目录	当前目录
AGENT815_PERMISSION	ask / auto / yolo	auto
AGENT815_MAX_CONTEXT	上下文窗口 tokens (压缩阈值=85%)	300000
AGENT815_MAX_STEPS	每 turn 步数预算	60
AGENT815_SUBAGENT_MAX_STEPS	子代理独立步数预算	30

3. 本机运行

```
# 交互模式 (/exit 退出, /memory 看记忆, /compact 手动压缩)
python -m agent815 --workdir /path/to/project

# 一次性任务
python -m agent815 --workdir ./proj --permission auto \
  --task "创建 utils.py 并写好 unittest, 跑通为止"

# 恢复会话 (重放 transcript 继续)
python -m agent815 --workdir ./proj --resume ./proj/.agent815/sessions/<ts>.jsonl \
  --task "继续: 把刚才的测试再补两个边界用例"
```

4. Docker (dev-server 已验证)

```
docker build -t 815agent:latest .
docker run --rm -v /tmp/ws:/workspace \
```

```
-e AGENT815_API_KEY=$KEY -e AGENT815_MODEL=qwen3.7-max \
-e AGENT815_PERMISSION=auto \
815agent:latest --task "写一个记账 CLI 并跑通 unittest"
```

容器内跑离线测试

```
docker run --rm --entrypoint python 815agent:latest -m unittest discover -s /app/tests
```

5. ACP 宿主模式

```
python -m agent815 --acp
# 每行一个 JSON-RPC 帧（stdout 只走协议，日志全在 stderr）：
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{}}
{"jsonrpc":"2.0","id":2,"method":"session/new","params":{"workdir":"/path"}}
{"jsonrpc":"2.0","id":3,"method":"session/prompt","params":{"sessionId":"...", "text":"任务..."}}
```

6. 扩展点

扩展	位置	说明
自定义技能	<code>skills/<name>/SKILL.md</code>	frontmatter 写 name/description，正文即操作手册；索引自动进 system prompt
进程外 hook	<code>.agent815/hooks.d/<event>_*.py</code>	stdin 收 JSON payload；stdout 打印 {"decision":"block","reason":"..."} 即阻断
新工具	<code>tools.py</code>	@tool 装饰器注册；记得在 sandbox.KNOWN_TOOLS 登记只读属性

7. 测试

```
python tests/test_all.py    # 35 个离线测试（FakeLLM，不触网）
```

对比报告

名称说明：教程对照的两个工业 codebase 是 **kimi-code** 与 **Hermes** (hermes-agent 0.18) 。
任务描述中的 "deepseek-harness" 在教程素材中不存在，本报告按 Hermes 对照；若指其他 DeepSeek 系 harness，下表维度（主循环形态/压缩策略/安全模型/记忆路线/状态模型）同样可直接套用。

定位：演化阶梯爬到哪一级

教程的方法论：每个模块从 50 行最小设计出发，每一级演化对应一类真实事故。**815agent 刻意停在"最小 + 第 1~2 级"**——它的价值不是功能齐全，而是标定出"一个能自动写代码的 agent 的最小完备集"，并且每个模块都被真实负载验证过（35 单测 + 8 个现场任务）。kimi-code 爬到"做对 + 做安全"，Hermes 爬到"做进系统"。

全模块对照表

模块	815agent	kimi-code	Hermes
主循环	while ask→act→append; max_steps 硬墙 + grace 收尾；工具错误回填	turn/step 队列 + 错误处理器插件 链（核心四行语义，重试/续跑全插件化）	预算驱动 while（90 迭代 + refund 记账 + grace call） + prologue/epilogue 分层
上下文	chars/4 估算；85% 单阈值； 头尾保留 + 免疫包裹摘要； fail-open	锚点实测+估算双层计量；0.85 触发 阻塞式 job；溢出三级收缩自愈	四阶段压缩 + 三重反抖动 （-1 哨兵/无效计数/600s 冷却） + 压缩锁 + in- place 保会话 id
工具	10 工具字典注册表；参数 校验；50K 落盘治理	三层折叠注册表 + toolPolicy×permissionPolicy 双 正交平面 + select_tools 渐进披露	AST 自动发现 + 核 心/bundle + 足迹 6 级阶 梯 + tool_search 桥接
沙盒权限	hardline 先于 yolo + NFKC/\$IFS 去混淆 + 三分	12 节静态权限链 + Tool(arg-glob) 规则 + 三档模式；微秒级零 LLM	8 层纵深：9 步去混淆 + hardline 12 条 + Smart

	级裁决 + workdir 围栏 + 子进程环境剥离		Approval (LLM 三态) + 6 类执行后端 + 网络出口隔离
记忆	MEMORY.md 冻结快照 + provenance 追加 + 注入安检 (多会话防污染加固后)	无跨会话记忆系统 (有意决策) : 会话即记忆 (wire journal + resume) + 声明式 AGENTS.md	双路注入 (冻结快照 + 每轮 prefetch 围栏) + provider ABC + fork 巩固 + 漂移检测
技能	frontmatter 索引常驻一行 + skill_view 按需正文 (两层披露)	双路激活 (用户斜杠/模型工具) + wire 幂等记账 + 压缩取舍 (用户 keep/模型 drop)	纯文档契约 + 紧闭环 fork (金路径 4 级) + 松闭环 curator (30/90 天流转, 永不硬删)
Hooks	5 事件回调表; before_tool/stop 可阻断; 进程外 30s 强杀; stop 一次性门锁	20 事件目录 + 进程外子进程 + zod 结构化输出 + Stop 续跑门锁	少而深: 三类同步 callback + 插件 hook 点 (接审批/压缩/学习闭环)
ACP	stdio JSON-RPC 分发表; stdout 纯协议; initialize/session 两方法	双包双宿主 + 纯函数转换表 + 客户端能力反向注入 (acp-fs/acp-terminal)	4-worker 线程池 + copy_context 并发隔离 + 双层审批 + 60s 超时 deny
会话/子代理	JSONL 追加重放 + delegate_task 隔离派生 (独立 messages/transcript/预算, 只回 distill)	wire 事件溯源 (单一一致性边界) + 扁平子代理 + swarm 突发 5 + 700ms 限速	SQLite 双 FTS5 (CJK trigram) + 软删归档 + session_search + fork 5 重隔离

行为差异 (可观测)

维度	815agent	kimi-code	Hermes
代码量	核心 ~1,300 行 (11 文件)	agent-core-v2 数万行 (loopService 单文件 1,222 行)	更大 (conversation_loop 5,294 行 / SessionDB 6,322 行)
单次交互延迟	非流式, 整段返回 (最小实现代价)	流式优先, 首 token 延迟是一等指标	事件回调外发, 按表面桥接

失控兜底	max_steps 硬墙 + stop 门锁	插件链（信任注册齐全）	预算硬墙 + refund + 显式退出诊断
无人值守能力	有限（ask 模式外无 LLM 兜底裁决）	低（设计前提：用户在终端前）	高（Smart Approval/超时 deny/网络出口，为无人值守设计）
跨会话学习	手动 save_lesson + 冻结快照（声明+轻自产）	零（外包给用户写规则）	全自动（fork 提取 + curator 生命周期）
可审计性	JSONL transcript 全量可查	wire.jsonl 事件溯源，全状态可重放	SQLite + FTS5 全文检索（含压缩归档行）

结论：什么时候选谁

- 815agent**：教学/二次开发基座。每个模块一眼看完，每行代码都能回答"防什么事故"；能力缺口（流式、子代理调度、反抖动、MCP）是有意留白，按教程演化阶梯"痛了再加"。
- kimi-code 型**：有人盯着的编程 CLI。延迟与确定性优先，复杂度投在"把功能做对/做安全"，记忆与学习外包给用户。
- Hermes 型**：无人值守多表面平台。安全边界与状态可恢复优先，每个模块都要"做进系统"，代价是数倍代码量。

问题与解决

问题一：多子代理上下文压缩污染

结论：815agent 不存在该污染 —— 加固前因为没有子代理；新增 delegate_task 后因为"物理隔离"设计。

这类污染是什么

教程记载的原型事故是 **Hermes #38727**：父 agent 与后台 fork **共享同一个 session**，两边同时触发压缩 → 各自写出一套压缩后历史 → 双 sibling 会话分叉。推而广之，凡"多执行体共享可变上下文/共享持久化"的子代理设计，都会踩三个坑：

- 父压缩把子代理的 tool_call/result 对从中间切开，协议直接 400；
- 两个执行体对同一历史各自压缩，账本分叉（谁的中段都不是对方的）；
- 子代理的探索日志全量回灌主线窗口，主线压缩被迫提前且摘要被探索噪声稀释。

kimi-code 的解法（参照来源）

- **扁平子代理 = 普通 Agent Scope**：显式注释"无父子嵌套"，每个 agent 有自己的 `wire.jsonl` —— 压缩是各自 wire 内的一个 Op，单一一致性边界，天然不互相碰；
- **只回传 distillSummary**：子代理探索过程不进主线，回传的摘要要有 minChars/重试纪律；
- **historySafeToCompact**：压缩应用前做前缀逐引用相等校验，发现历史被并发篡改就放弃应用；
- **swarm 限速**：突发 5 + 700ms/个，防并行派生触发 provider 限速雪崩。

815agent 的落地（subagent.py, 47 行）

delegate_task(task)

- └ 全新 Agent 实例：独立 messages（物理隔离 → 父压缩永远碰不到子上下文）
- └ 独立 transcript 文件（kimi 的"每 agent 一个 wire.jsonl"等价物）
- └ 独立预算 AGENT815_SUBAGENT_MAX_STEPS=30（不侵蚀父步数）
- └ 独立 stop 门锁
- └ system prompt 与父同前缀（暖缓存，kimi fork 继承 _cached_system_prompt 同思路）
- └ 只回传最终文本（distill, 10K 截断）——探索日志不灌主线

✓ 验证：T7 现场任务中父代理 transcript 仅含 `delegate_task → write_file → bash×2`；子代理的 11 次探索调用（list_dir/read_file/glob）完整记录在自己独立的 transcript 文件里，主线窗口零污染。单测 `TestSubagent` ×3 覆盖隔离/预算/蒸馏截断。

问题二：多会话记忆 (memory) 压缩污染

结论：加固前存在两类真实风险（并发写竞态、带毒注入），一类天然免疫（摘要劫持）；已全部修复并测试。

形态 A：压缩把记忆卷进摘要 → 摘要劫持（设计天然免疫）

记忆快照位于 `messages[0]` (system)，压缩的 head 保护 (`messages[:2]`) 保证它**永不进入被摘要的中段**；摘要输出又带 `[REFERENCE ONLY]` + 最新用户消息优先 免疫包裹。两条防线叠加，Hermes #11475/#35344 类事故在 815agent 结构上不成立。

形态 B：并发会话写 MEMORY.md → 截断竞态（已修复）

原风险：两个会话同 workdir 同时超限截断 → 读-改-写竞态，丢教训甚至写出半文件；教训无时间戳，无法溯源是谁写的。

修复：① 追加写带 UTC provenance (`- [2026-08-15T07:30:12Z] ...`)，保持行级原子；② 截断重写改 **tmp+rename 原子替换** (kimi AppendLogStore 的 cutover 思路最小化)。

形态 C：被污染条目随快照带毒注入（已修复）

原风险：某会话被提示注入后调 `save_lesson` 写入"忽略之前所有指令..."之类的毒条目 → 之后**每个新会话**都随冻结快照自动注入（自学习路线的固定税，Hermes #11475 系）。

修复：快照加载时逐行**注入安检**（指令覆盖/角色冒充/系统标签/压缩围栏回显等模式），命中条目替换为占位符 `[已过滤：疑似注入内容的记忆条目]` —— Hermes threat_patterns 扫描的最小对应物。

验证：单测 `test_injection_scan_on_load` 确认毒条目被过滤、正常条目不受影响。

为什么不干脆学 kimi 不要记忆？

kimi-code 的答案是"会话即记忆 + 用户声明规则"，把跨会话学习整体外包——零污染面，但也零自学习。815agent 保留轻量自产记忆（agent 现场两次主动 `save_lesson` 都写出了有价值的教训），代价是必须支付上述 B/C 两项防御——这是教程第 5 章"自学习固定税"的最小缴纳方式。

实测发现的全部问题与修复（时间序）

#	问题	根因	修复	验证
1	一次性任务结束打印崩溃	Windows 控制台 GBK，模型输出含  等字符	cli.py 启动即 reconfigure stdout/stderr 为 utf-8/replace	冒烟 exit 0
2	进程外 hook 读 payload 失败	子进程 sys.stdin 默认 GBK	示例 hook 改 buffer=utf-8；父进程注入 PYTHONIOENCODING=utf-8	T4b 阻断适配实测
3	单 user 会话永不压缩	"最后 user 必须在 tail"守护把 cut 拉回 1 → middle 恒空	守护仅在 last_user_idx ≥ 2 时生效 + 回归测试	t6d 现场压缩触发 2 次
4	记忆并发写竞态 / 带毒注入	见形态 B/C	provenance + tmp+rename + 注入安检	TestMemory ×3
5	无子代理机制（能力缺口）	最小设计留白	delegate_task 物理隔离派生	TestSubagent ×3 + T7 现场

已知边界（有意不修，对应教程演化阶梯）

边界	对应演化级	什么时候该加
极小窗口下压缩 thrash（压完又超阈）	Ch.2 第 6 级反抖动三件套	真实长会话逼近 30 万窗口时
LLM 非流式（整段等待）	Ch.1 第 1 级流式解析	交互体验成为瓶颈时
无并发子代理调度（swarm）	Ch.9 第 5 级批调度限速	并行探索需求出现时
无 MCP 外部工具桥接	Ch.3 第 8 级	需要接外部生态时
无 LLM 兜底裁决（Smart Approval）	Ch.4 第 10 级	无人值守部署时

工程问答

一面 · 工程细节题

Q1上下文用的是 message window 还是 token window? 线上为什么这么选?

已有机制 **token window**。 `context.py` 以估算 token 数 (chars/4) 对 30 万窗口设 85% 触发线, message window 无法表达"10 条消息里有一条 240KB 的工具输出"这种真实分布——T5 实测单条 read_file 结果就是 25 万字符。选 token window 的代价是估算漂移, 工业解法是 kimi 的"实测锚点+估算尾部"双层计量, 815agent 停在单层粗估 + 85% 保守阈值 + 溢出自愈兜底 (Q4)。

kimi: 双层计量 · Hermes: 三套信号协调

Q2会话持久化的过期策略怎么定? 遇到过并发会话冲突吗?

针对性补全 存储选型是 **JSONL 文件而非 Redis** (教程准则四: 只要恢复→追加日志够用, 要上检索才换库)。过期策略: `session.cleanup_old_sessions(workdir, ttl_days=30)`, 按文件 mtime TTL 清理, CLI `--cleanup-days N`。

并发冲突遇到过, 分两处解决: ① 多会话同 workdir 写 MEMORY.md → 行级原子追加 + tmp+rename 原子替换截断; ② 多子代理共享上下文压缩 → 物理隔离 (各自独立 transcript), 参考 Hermes #38727 的尸检。 **kimi: AppendLogStore tmp+rename cutover · Hermes: SQLite 压缩锁 TTL 300s**

Q3长期记忆用向量库吗? 记忆越来越多、检索越来越不准怎么办?

针对性补全 **不用向量库**——815agent 的记忆是有 8000 字符硬上限的平文件 (容量即精度: 上限强制淘汰旧条目, 检索域永远小到关键词足够准)。本轮补了 `memory_search` 工具 (关键词检索, 只读)。真正的防线在写入侧: provenance 时间戳溯源 + 注入安检, 毒条目不进检索域。什么时候该上向量库: 记忆条目破千、跨项目复用时——教程里 Hermes 用 SQLite FTS5 (含 CJK trigram) 也是先 BM25 后向量, 且曾砍掉摘要 LLM 路径回归纯 BM25。 **Hermes: FTS5+trigram, "session 霸榜"教训**

Q4上下文 token 溢出的兜底方案是什么?

针对性补全 三层: ① 预防——每步 preflight 估算超 85% 即压缩; ② **自愈 (本轮新增)**——provider 报 context/overflow/length 错误时 `_chat()` 捕获, `force=True` 强制压缩后重试, 连续 3 次放弃 (对齐 kimi 溢出自愈上限); ③ 兜底——摘要调用失败 fail-open 保原会话, 预算耗尽还有 grace 收尾。单测 `TestOverflowHeal` ×2 覆盖成功与放弃两条路径。 **kimi: 错误处理器认领 413, 下调有效窗口 ×0.85**

Q5单用户日均 token 成本多少? 百万级流量怎么控制?

针对性补全 可测才能答: 本轮给每步 LLM 调用加了**用量账本** (prompt/completion/calls 累加, turn 结束落 transcript usage 事件)。现场实测: 一次"建文件+运行+查记忆"的轻任务 = **5,838 prompt + 189 completion / 4 次调用**; 一次完整项目任务 (T1, 39 测试项目) 约 14 次工具调用、~10 次 LLM 调用。百万级控制三板斧全在架构里: ① 缓存宪法 (前缀只追加, DashScope 命中缓存约 1/10 价); ② 结果治理 (50K 落盘只回 2K, T5 实测 25.6 万字符→2,130); ③ 预算硬墙 (max_steps + 子代理独立预算)。 **kimi: 分段缓存+末尾注入 · Hermes: 4 断点+冻结快照**

Q6调用拦截: 用户恶意刷长文本怎么限制?

针对性补全 三层拦截: ① 入口——`AGENT815_MAX_INPUT_CHARS` (默认 20 万字符) 超限直接拒绝, **不消耗任何 LLM 调用** (单测验证 FakeLLM 零调用); ② 工具侧——bash 120s 超时、输出 30K 截断、

大结果 50K 落盘；③ 循环侧——max_steps 硬墙 + stop hook 一次性门锁防"永不结束"。

Q7为什么用基座模型不用微调？拿什么数据做判断？

设计决策 选基座 (qwen3.7-max) 因为 Harness 架构把行为约束都放在了**工程层** (system prompt 纪律 + 工具协议 + 权限链)，这些不需要权重级定制；微调的适用面是输出格式/领域风格稳定且高频的场景，coding agent 的需求是通用推理 + 工具调用可靠性。判断数据：① 工具调用成功率 (实测 73 次调用 0 协议错误)；② 任务完成率 (8 个现场任务 8/8 exit 0)；③ 失败自愈率 (T1/T3 测试失败 → edit_file 修复 → 复测通过)；④ 单任务 token 成本 (Q5 账本)。微调唯一可能值回票价点是把 BASE_PROMPT 的纪律烧进权重省 system prompt token——但会损失迭代速度，不划算。

Q8讲一个线上故障排查：改了哪块逻辑，效果多少？

真实案例 **单 user 会话压缩失效**。现象：小窗口压力测试 (MAX_CONTEXT=1500) 下压缩事件恒为 0。排查：逐步重放真实 transcript 计算各 preflight 时点估算，确认第 3 条消息起已超阈值却不触发 → 定位到 `context.py` "最后一条 user 必须在 tail" 守护规则：一次性任务唯一 user 消息在 head (索引 1)，守护把 cut 强制拉回 1，middle 恒空，压缩静默放弃。修复：守护仅在 `last_user_idx ≥ 2` 生效。效果：同任务压缩触发从 **0 → 2 次**，免疫摘要正确注入；回归测试 `test_single_user_message_still_compacts` 防复发。教训：守护规则必须考虑"被保护对象已在保护区"的边界。

二面 · 系统设计题

Q9系统模块怎么拆分？职责与通信？为什么这么拆？

已有机制 11 个模块按"窄腰核心 + 边上插拔"拆 (见模块设计页)：loop 是唯一有状态核心，其余模块通过三个窄接口通信——① 工具协议 (schema+handler 字典)；② 事件表 (hooks 5 事件)；③ 消息列表 (唯一共享状态，只追加)。拆分依据是教程两条宪法：核心小到可以穷举测试 (loop 168 行)，换 provider/换存储/换表面不动核心。通信不走消息总线而共享不可变前缀的消息列表，是为了缓存宪法——任何总线式动态注入都会打碎前缀缓存。

Q10怎么判断单工具简单任务 vs 多步拆解复杂任务？拆解粒度？

已有机制 815agent 走 kimi 路线：**拆解权交给模型，工程层只设边界**——模型自主决定是否连续调工具 (终止由模型决定)，工程层的控制是：max_steps 硬墙 (60)、子代理独立预算 (30)、`delegate_task` 让模型把"探索性子任务"显式派生出去 (粒度信号：会污染主线上下文的活就派生，探索日志只回蒸馏报告)。Hermes 路线是预算 refund (模型写代码循环调工具不侵蚀预算)——两种都验证了"不要在工程层替模型做拆解判断"，那会变成永远追不上模型能力的启发式。

Q11任务执行一半失败：重试、回滚还是重新规划？触发条件？

已有机制 分四层，每层触发条件显式：

失败类型	策略	触发条件
------	----	------

工具执行失败	错误文本回填， 模型自行重试或换路 （T1 实测：测试失败→edit_file→复测通过）	任何 tool exception
429/5xx/网络超时	指数退避重试（×2 上限 30s + 20% 抖动 + Retry-After），最多 5 次	HTTP 状态码/连接异常
上下文溢出	强制压缩后重试，连续 3 次放弃	错误文本含 context/overflow/length
权限拒绝/hook 阻断	不重试不绕过 ，原因回填让模型换方案（T4b 实测 pip 被禁→标准库完成）	sandbox/hook veto

回滚在最小设计里刻意没有（文件操作无事务），对应演化级是 kimi 的 undo/wire Op 重放——痛了再加。

Q12多工具怎么联动？权限怎么管控？怎么防越权？

已有机制 联动：工具是纯字典注册表，联动顺序由模型按 schema 描述编排；结果治理保证任何单工具输出不灌爆窗口。权限五道防线，顺序即优先级：① **hardline 硬线**（rm -rf /、fork bomb、写 authorized_keys，**yolo 也不放行**——用户不能授权无恢复路径的破坏）；② 去混淆（NFKC 全角/\$IFS/引号拼接）防绕过；③ 未知工具 fail-closed；④ 只读/写执行三分级 + ask/auto/yolo 模式；⑤ workdir 围栏 + 子进程环境变量剥离（KEY/TOKEN 类，实测工作区零密钥痕迹）。防越权的价值观：**危险检测先于用户授权**。 **Hermes: hardline 先于 yolo bypass · GHSA-rhgp 凭据隔离**

三面 · 认知题（回答框架，以本项目为论据）

Q13Agent 是长期风口还是短期热点？长期落地方向？

长期。论据：模型能力每代跃迁，但教程里两个工业 codebase 的厚度证明**工程表现的上下限由 Harness 决定**——同一模型接不同 Harness，安全性/成本/长任务存活率差一个数量级，这层工程不会随模型变强而消失（缓存宪法、权限纵深、状态可恢复都是模型外问题）。落地方向：有人值守的编程 CLI（kimi 型）先跑通，无人值守的多表面平台（Hermes 型）跟随，自学习闭环（记忆/技能双闭环）是拉开差距的下半场。

Q14和普通大模型开发者比，核心竞争力？

能从"事故"反推架构。815agent 每个模块都能回答"防什么事故、对应演化阶梯第几级、kimi/Hermes 怎么解"——demo 开发者堆框架，工程师管爆炸半径（hardline 先于 yolo）、管成本（缓存宪法）、管最坏情况下限（fail-open 方向都是显式决策）。

Q151-2 年内最大瓶颈是模型能力还是工程能力？ToB 还是 ToC 先跑通？

工程能力。模型已经"够用"（实测 qwen3.7-max 零协议错误完成 8 个完整项目），卡脖子的是：长会话成本（压缩与缓存的博弈）、无人值守安全（纵深防御）、状态可恢复（断点续跑）。ToB 先跑通——容错预算高、场景边界清晰、权限模型可预先声明；ToC 的越权风险面不可控。

Q16从 0-1 带一条 Agent 业务线，第一步做什么？

先定形态再写代码（教程决策树 Q1）：有人盯着还是无人值守？这一个问题决定主循环形状（插件链 vs 预算硬墙）、安全厚度（规则链 vs 纵深栈）、存储选型（JSONL vs SQLite+FTS5）。招人按模块招（循环/上下文/安全/记忆技能/表面），每人负责一条演化阶梯的爬升路线，评审标准就是"每份复杂性答得出防什么事故"。